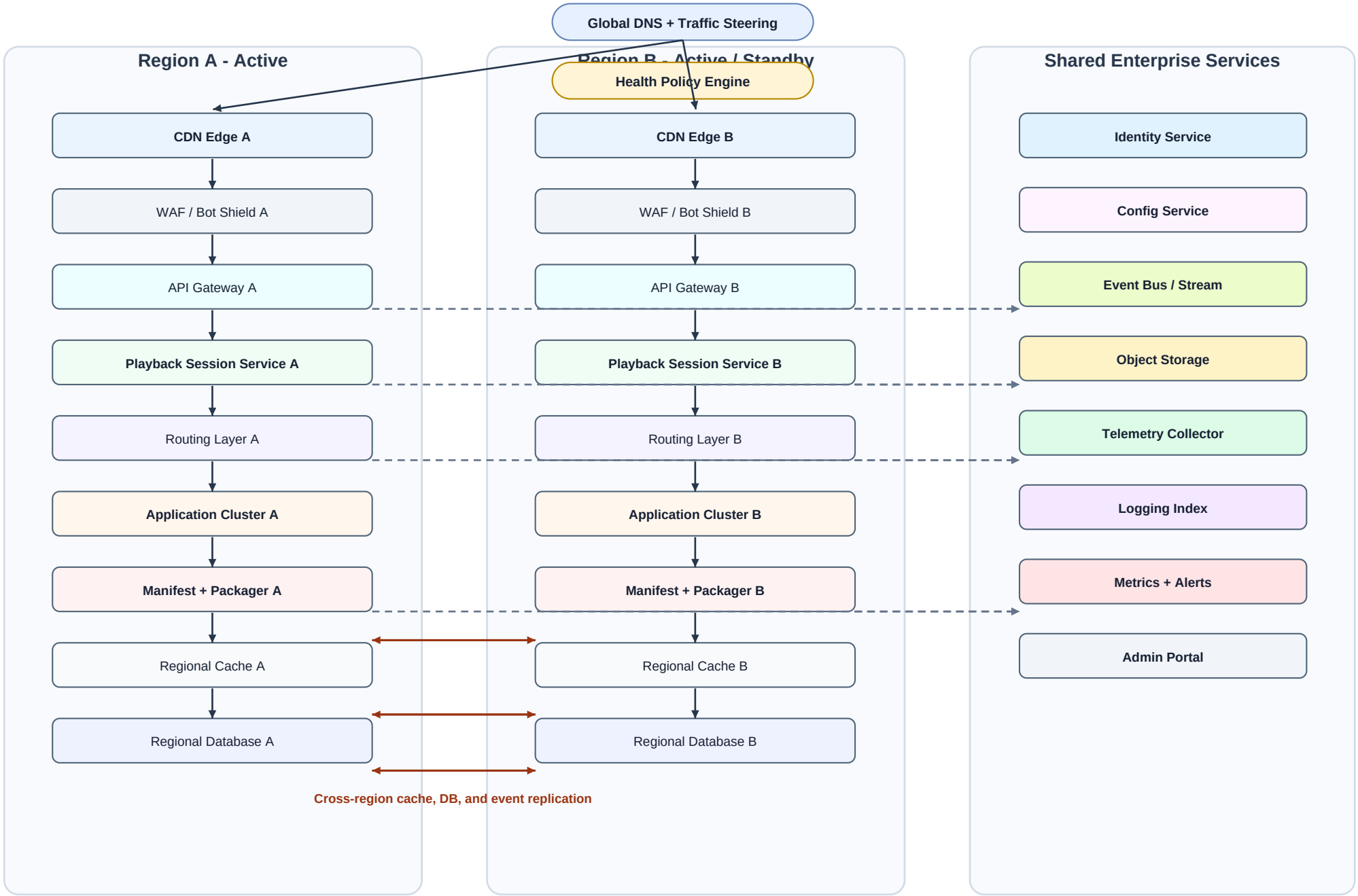
Synthetic Multi-Region Streaming Platform - Architecture Stress Sample

Synthetic multi-region architecture; no real company names or proprietary logs



Solid arrows = request / response path. Dashed arrows = shared-service dependency. Brown arrows = cross-region sync / failover signals.

Journey Map - Multiple Rails and Shared Nodes

Designed to test Journey Discovery, Shared Nodes, Multi-Rail, Bidirectional Context, Learning Chapters

Primary Playback Request	Client -> DNS -> CDN -> WAF -> Gateway -> Session -> Routing -> App Cluster -> Manifest/Packager -> CDN -> Client
Authentication / Validation	Gateway -> Identity Service -> Policy Check -> Session Token -> Gateway
Content Delivery	Manifest/Packager -> Object Storage -> Regional Cache -> CDN Edge -> Client
State Sync	Regional DB A <-> Regional DB B; Cache A <-> Cache B; Event Bus replication
Control / Config	Admin Portal -> Config Service -> Routing Layer -> Application Cluster
Observability	Every region -> Telemetry Collector -> Logging Index -> Metrics + Alerts -> Health Policy Engine
Failover	Health Policy Engine -> Global DNS -> shift traffic from Region A to Region B

Why this sample is useful

1. Same component name appears in Region A and Region B, forcing region-aware identity handling.
2. Identity, Config, Event Bus, and Observability are shared across several journeys.
3. State sync is bidirectional and should not be mistaken for the primary request path.
4. Failover creates a conditional alternate path without becoming the default path.
5. The curriculum should group focuses into Overview, Journey, Responsibilities, Components, Handoffs, Recap.

Component Glossary and Expected Generic Roles

No vendor names, no real infrastructure names

Component Pattern	Expected Role	Why it exists in the architecture
Global DNS / Traffic Steering	entry / control	Selects a region based on health, policy, and proximity.
CDN Edge	entry / delivery	Receives playback requests and delivers cached objects when available.
WAF / Bot Shield	validation	Applies coarse request validation before application handling.
API Gateway	entry / control	Central routing and policy boundary for application APIs.
Playback Session Service	processing / state hint	Creates or validates playback session context.
Routing Layer	control	Chooses regional application or service destination.
Application Cluster	processing	Runs core business logic for playback and catalog decisions.
Manifest + Packager	processing / delivery	Builds playable manifests and packaging responses.
Regional Cache	state / delivery	Stores hot responses close to regional services.
Regional Database	state	Persistent regional system of record.
Identity Service	auth / validation	Shared authentication and token validation.
Config Service	control	Shared control plane for feature flags and routing policies.
Event Bus / Stream	processing / observability	Carries asynchronous events across services and regions.
Telemetry Collector	observability	Receives metrics, logs, and trace signals.
Metrics + Alerts	observability	Summarizes health and raises failover signals.

Expected Regression Targets for Cachey Notebook

Use this PDF before BUG-17+ to prevent overfitting to simple diagrams

Expected Journeys

- primary_request_journey
- content_delivery_journey
- auth_validation_journey
- control_configuration_journey
- state_sync_journey
- observability_journey
- failover_journey

Expected Hard Cases

- Duplicate regional components: CDN Edge A/B, API Gateway A/B, Database A/B
- Shared nodes: Identity, Config, Event Bus, Telemetry, Metrics
- Bidirectional state sync and event replication
- Conditional failover rail that should not replace primary path
- Async event journey separate from synchronous playback path

Expected Chapter Health

- learning-chapters.health.valid = true
- duplicateFocusAssignmentCount = 0
- unassignedFocusCount = 0
- missingRequiredChapterCount = 0
- traversalChanged = false

Demo Safety Rules

- All names are generic and synthetic
- No real company, partner, internal service, IP, host, log, schema, metric, or incident data
- Architecture is inspired by common enterprise patterns, not a copied proprietary diagram

Canonical request path to look for:

Client -> Global DNS -> Region A CDN -> WAF -> API Gateway -> Playback Session -> Routing -> Application Cluster -> Manifest/Packager -> Regional Cache -> CDN -> Client